CLAUDE.md — Módulo de Seguros Agéntico para atiende.ai

Qué es este proyecto

Módulo que se AGREGA al monorepo existente de atiende.ai. Permite a agentes de seguros mexicanos cotizar automáticamente en 15+ aseguradoras simultáneamente via browser automation AI, entregar comparativas por WhatsApp en <90 segundos, gestionar pólizas, renovaciones y cobros — todo 100% autónomo. Modelo: $2,999-$6,999 MXN/mes por agente. Deploy: Vercel (frontend+API) + Railway (Python workers con browsers).

Arquitectura General

```
Cliente WhatsApp ──mensaje──▶ Meta Cloud API Webhook
                                  |
                                  ▼
                        ┌──────────────────┐
                        |  Next.js API Route   |
                        |  /api/insurance/     |
                        |  intake              |
                        └──────────┬──────────┘
                                   |
                                   ▼
                  ┌──────────────────────────┐
                  |  AI Data Extractor (LLM)    |
                  |  Gemini 2.5 Flash via       |
                  |  OpenRouter — extrae JSON   |
                  |  estructurado del mensaje   |
                  └──────────┬───────────────┘
                             |
                             ▼
                  ┌──────────────────────────┐
                  |  QStash Fan-Out             |
                  |  Dispara N jobs paralelos   |
                  |  uno por cada aseguradora   |
                  └──┬──────┬──────┬──────┬──┘
                     |      |      |      |
                  ┌──▼──┐ ┌──▼──┐ ┌──▼──┐ ┌──▼──┐
                  |Qual. | | GNP  | | AXA | | HDI | ...x15
                  |Worker| |Worker| |Work.| |Work.|
                  └──┬──┘ └──┬──┘ └──┬──┘ └──┬──┘
                     |       |       |       |
            Railway Python Workers (Playwright+Skyvern)
            Cada uno abre browser, login, cotiza, extrae
                     |       |       |       |
```

```
└──────┴──────┴──────┘
              │
              ▼
┌──────────────────────────┐
│  QStash Callback         │
│  /api/insurance/callback │
│  Guarda en Supabase      │
│  Publica SSE progress    │
│  Envía WhatsApp progresivo │
└──────────────────────────┘
              │
              ▼
┌──────────────────────────┐
│  Supabase (PostgreSQL)   │
│  + Supabase Storage (PDFs) │
│  + Supabase Vault (creds) │
└──────────────────────────┘
```

Stack Tecnológico

Componente	Tecnología	Razón
Frontend	Next.js 15 + shadcn/ui + Tailwind	Ya en atiende.ai
API	Next.js API Routes (App Router)	Monorepo, edge-ready
Workers	Python 3.12 + FastAPI + Playwright	Browser automation necesita Python
AI Browser	Browser Use 0.12 + Skyvern SDK	Dual-layer: rápido + resiliente
Database	Supabase PostgreSQL	Ya en atiende.ai, RLS, Vault
Queue	Upstash QStash	Serverless fan-out, callbacks
Cache/State	Upstash Redis	Circuit breaker, SSE state
LLM	Gemini 2.5 Flash (OpenRouter)	Extraction rápida y barata
WhatsApp	Meta Cloud API v21	Ya en atiende.ai
Cron	Vercel Cron + QStash schedules	Renovaciones, pagos
Storage	Supabase Storage	PDFs de cotizaciones
Encryption	AES-256-GCM (Node.js crypto)	Credenciales de portales
Monitoring	Sentry	Ya en atiende.ai

Componente	Tecnología	Razón
Deploy FE	Vercel	Ya en atiende.ai
Deploy Workers	Railway (Docker)	Long-running browsers

Reglas para Claude Code

1. **SIEMPRE pregunta antes de actuar** — Pide confirmación, API keys, credenciales
2. **Un paso a la vez** — Sigue las fases EN ORDEN ESTRICTO
3. **Este módulo se AGREGA** — No toques archivos existentes de atiende.ai a menos que sea para importar/registrar rutas nuevas
4. **Prefijo `ins_`** — Todas las tablas nuevas llevan prefijo `ins_` para no colisionar
5. **RLS en TODO** — Cada tabla nueva debe tener Row Level Security
6. **Credenciales SIEMPRE encriptadas** — AES-256-GCM, nunca plaintext ni en logs
7. **Error handling en TODO** — try/catch, logging estructurado, graceful degradation
8. **TypeScript estricto** — No `any`, usa tipos definidos en `types.ts`
9. **Archivos máximo 300 líneas** — Si crece más, dividir en módulos
10. **Tests después de cada fase** — No avances sin checkpoint verde

---

FASES DE IMPLEMENTACIÓN

Sigue estas fases EN ORDEN. No saltes ninguna.

FASE 0: Setup — Estructura y Dependencias

Objetivo: Crear la estructura de archivos del módulo de seguros dentro del monorepo existente sin romper nada.

Dependencias: atiende.ai base deployada y funcionando.

Qué pedir al usuario:

- [ ] "¿Tu atiende.ai está corriendo localmente con `npm run dev`? Confirma."
- [ ] "Voy a instalar dependencias nuevas. ¿Continúo?"

Paso 1 — Instalar dependencias:

```bash
```

```
npm install @upstash/qstash @upstash/redis p-limit p-retry p-timeout pdf-lib crypto-
```

Paso 2 — Crear estructura de carpetas:

```bash
mkdir -p src/lib/insurance/agents
mkdir -p src/lib/insurance/utils
mkdir -p src/lib/insurance/workers
mkdir -p src/app/\(dashboard\)/insurance/quotes
mkdir -p src/app/\(dashboard\)/insurance/policies
mkdir -p src/app/\(dashboard\)/insurance/carriers
mkdir -p src/app/api/insurance/quote
mkdir -p src/app/api/insurance/carriers
mkdir -p src/app/api/insurance/credentials
mkdir -p src/app/api/insurance/callback
mkdir -p src/app/api/insurance/stream
mkdir -p src/app/api/cron/insurance
```

Paso 3 — Actualizar `.env.local` (agregar estas líneas al final):

```env
```

```
# === MÓDULO SEGUROS ===
# Upstash QStash (https://console.upstash.com/qstash)
QSTASH_TOKEN=
QSTASH_CURRENT_SIGNING_KEY=
QSTASH_NEXT_SIGNING_KEY=

# Upstash Redis (https://console.upstash.com/redis)
UPSTASH_REDIS_REST_URL=
UPSTASH_REDIS_REST_TOKEN=

# Skyvern (https://app.skyvern.com/settings)
SKYVERN_API_KEY=

# Railway Workers URL (se llena después del deploy de workers)
INSURANCE_WORKER_URL=
INSURANCE_WORKER_SECRET=generame_un_secret_random_aqui

# Credential encryption (genera con: openssl rand -hex 32)
CREDENTIAL_ENCRYPTION_KEY=

# App URL (para QStash callbacks)
APP_URL=http://localhost:3000
```

Paso 4 — Crear archivos base:

`src/lib/insurance/types.ts`:

```typescript
```

```
// Tipos compartidos del módulo de seguros

export type InsuranceLine = 'auto' | 'vida' | 'gastos_medicos' | 'hogar' | 'negocio'
export type CoverageType = 'amplia' | 'limitada' | 'rc_obligatoria'
export type QuoteStatus = 'pending' | 'validating' | 'quoting' | 'partial' | 'comple
export type IndividualQuoteStatus = 'pending' | 'running' | 'success' | 'declined' |
export type PolicyStatus = 'active' | 'pending_payment' | 'cancelled' | 'expired' |
export type PaymentStatus = 'pending' | 'paid' | 'overdue' | 'cancelled'
export type CarrierPortalType = 'browser' | 'api' | 'email'
export type CarrierHealthStatus = 'healthy' | 'degraded' | 'down'

export interface Carrier {
  id: string
  name: string
  slug: string
  logo_url: string | null
  portal_url: string
  portal_type: CarrierPortalType
  supported_lines: InsuranceLine[]
  market_share_auto: number
  health_status: CarrierHealthStatus
  is_active: boolean
}

export interface CarrierCredential {
  id: string
  tenant_id: string
  carrier_id: string
  agent_number: string | null
  is_active: boolean
  last_login_success: string | null
  login_failure_count: number
}

export interface QuoteRequestInput {
  insurance_line: InsuranceLine
  client: {
    name: string
    phone?: string
    email?: string
    rfc?: string
    birthdate?: string
    gender?: 'M' | 'F'
```

```
    zip_code: string
  }
  vehicle?: {
    brand: string
    model: string
    year: number
    version?: string
    use: 'particular' | 'comercial'
  }
  coverage_type?: CoverageType
  source: 'whatsapp' | 'web' | 'voice' | 'api'
  conversation_id?: string
  contact_id?: string
  raw_input?: string
}

export interface QuoteResult {
  carrier_slug: string
  carrier_name: string
  status: IndividualQuoteStatus
  annual_premium?: number
  monthly_premium?: number
  deductible_amount?: number
  deductible_percentage?: number
  coverages?: Array<{
    name: string
    sum_insured?: number
    included: boolean
  }>
  quote_number?: string
  valid_until?: string
  pdf_url?: string
  screenshot_url?: string
  duration_ms?: number
  error_message?: string
  error_type?: string
}

export interface QuoteProgress {
  request_id: string
  total: number
  completed: number
  failed: number
  results: QuoteResult[]
```

```typescript
  status: QuoteStatus
  best_price?: number
}

export interface WorkerQuotePayload {
  request_id: string
  tenant_id: string
  carrier_slug: string
  carrier_portal_url: string
  carrier_portal_type: CarrierPortalType
  insurance_line: InsuranceLine
  client_data: QuoteRequestInput['client']
  vehicle_data?: QuoteRequestInput['vehicle']
  coverage_type?: CoverageType
  credentials: {
    username: string
    password: string
    agent_number?: string
  }
}
```

`src/lib/insurance/constants.ts`:

```typescript
```

```
export const CARRIER_SEEDS = [
  { name: 'Qualitas', slug: 'qualitas', portal_url: 'https://agentes.qualitas.com.mx
  { name: 'GNP Seguros', slug: 'gnp', portal_url: 'https://intermediarios.gnp.com.mx
  { name: 'AXA Seguros', slug: 'axa', portal_url: 'https://distribuidores.axa.com.mx
  { name: 'HDI Seguros', slug: 'hdi', portal_url: 'https://portalagentes.hdi.com.mx'
  { name: 'Chubb Seguros', slug: 'chubb', portal_url: 'https://agentes.chubb.com/mx'
  { name: 'BBVA Seguros', slug: 'bbva', portal_url: 'https://api.bbva.com', portal_t
  { name: 'Zurich Seguros', slug: 'zurich', portal_url: 'https://portalagentes.zuric
  { name: 'Mapfre', slug: 'mapfre', portal_url: 'https://agentes.mapfre.com.mx', por
  { name: 'Seguros Atlas', slug: 'atlas', portal_url: 'https://portal.segurosatlas.c
  { name: 'AIG Seguros', slug: 'aig', portal_url: 'https://agentes.aig.com.mx', port
  { name: 'Banorte Seguros', slug: 'banorte', portal_url: 'https://seguros.banorte.c
  { name: 'Afirme Seguros', slug: 'afirme', portal_url: 'https://agentes.afirme.com'
  { name: 'SURA', slug: 'sura', portal_url: 'https://agentes.segurossura.com.mx', po
  { name: 'MetLife', slug: 'metlife', portal_url: 'https://agentes.metlife.com.mx',
  { name: 'Allianz', slug: 'allianz', portal_url: 'https://agentes.allianz.com.mx',
] as const

export const QUOTE_TIMEOUT_MS = 120_000  // 2 min por carrier
export const QUOTE_CACHE_TTL_HOURS = 4
export const MAX_CONCURRENT_QUOTES = 8
export const CIRCUIT_BREAKER_THRESHOLD = 5  // failures antes de abrir
export const CIRCUIT_BREAKER_TIMEOUT_MS = 300_000  // 5 min en open state

export const COVERAGE_LABELS: Record<string, string> = {
  amplia: 'Cobertura Amplia',
  limitada: 'Cobertura Limitada',
  rc_obligatoria: 'Responsabilidad Civil Obligatoria',
}

export const INSURANCE_LINE_LABELS: Record<string, string> = {
  auto: 'Seguro de Auto',
  vida: 'Seguro de Vida',
  gastos_medicos: 'Gastos Médicos Mayores',
  hogar: 'Seguro de Hogar',
  negocio: 'Seguro de Negocio',
}
```

Checkpoint Fase 0:

- [ ] `npm run build` pasa sin errores
- [ ] Carpetas creadas en `src/lib/insurance/` y `src/app/`

- [ ] `types.ts` y `constants.ts` importables sin error: `import { Carrier } from '@/lib/insurance/types'`
- [ ] `.env.local` tiene todas las variables nuevas (aunque vacías)

FASE 1: Base de Datos — Schema SQL Completo

Objetivo: Crear TODAS las tablas del módulo de seguros en Supabase.

Dependencias: Fase 0 completa.

Qué pedir al usuario:

- [ ] "Voy a ejecutar SQL en tu Supabase. Abre el SQL Editor en dashboard.supabase.com. ¿Listo?"

Crear archivo `supabase/insurance-schema.sql`:

```sql
```

```sql
-- ============================================================
-- MÓDULO DE SEGUROS: SCHEMA COMPLETO
-- Ejecutar en Supabase SQL Editor
-- Prefijo: ins_ para todas las tablas
-- ============================================================

-- 1. CARRIERS (catálogo de aseguradoras)
CREATE TABLE IF NOT EXISTS ins_carriers (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name TEXT NOT NULL,
  slug TEXT UNIQUE NOT NULL,
  logo_url TEXT,
  portal_url TEXT NOT NULL,
  portal_type TEXT NOT NULL DEFAULT 'browser' CHECK (portal_type IN ('browser', 'api
  supported_lines TEXT[] DEFAULT '{}',
  market_share_auto DECIMAL(5,2) DEFAULT 0,
  health_status TEXT DEFAULT 'healthy' CHECK (health_status IN ('healthy', 'degraded
  failure_rate_24h DECIMAL(5,2) DEFAULT 0,
  is_active BOOLEAN DEFAULT true,
  last_health_check TIMESTAMPTZ,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- Seed: Top 15 aseguradoras México
INSERT INTO ins_carriers (name, slug, portal_url, portal_type, supported_lines, mark
('Qualitas', 'qualitas', 'https://agentes.qualitas.com.mx', 'browser', '{auto}', 32.
('GNP Seguros', 'gnp', 'https://intermediarios.gnp.com.mx', 'browser', '{auto,vida,g
('AXA Seguros', 'axa', 'https://distribuidores.axa.com.mx', 'browser', '{auto,vida,g
('HDI Seguros', 'hdi', 'https://portalagentes.hdi.com.mx', 'browser', '{auto,vida,ho
('Chubb Seguros', 'chubb', 'https://agentes.chubb.com/mx', 'browser', '{auto,vida,ga
('BBVA Seguros', 'bbva', 'https://api.bbva.com', 'api', '{auto,vida,hogar}', 5.2),
('Zurich Seguros', 'zurich', 'https://portalagentes.zurich.com.mx', 'browser', '{aut
('Mapfre', 'mapfre', 'https://agentes.mapfre.com.mx', 'browser', '{auto,vida,gastos_
('Seguros Atlas', 'atlas', 'https://portal.segurosatlas.com.mx', 'browser', '{auto,v
('AIG Seguros', 'aig', 'https://agentes.aig.com.mx', 'browser', '{auto,vida,negocio}
('Banorte Seguros', 'banorte', 'https://seguros.banorte.com/agentes', 'browser', '{a
('Afirme Seguros', 'afirme', 'https://agentes.afirme.com', 'browser', '{auto,vida}',
('SURA', 'sura', 'https://agentes.segurossura.com.mx', 'browser', '{auto,vida,gastos
('MetLife', 'metlife', 'https://agentes.metlife.com.mx', 'browser', '{vida,gastos_me
('Allianz', 'allianz', 'https://agentes.allianz.com.mx', 'browser', '{auto,vida,gast
ON CONFLICT (slug) DO NOTHING;
```

```sql
-- 2. CARRIER CREDENTIALS (encriptadas, por tenant)
CREATE TABLE IF NOT EXISTS ins_carrier_credentials (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
  carrier_id UUID NOT NULL REFERENCES ins_carriers(id) ON DELETE CASCADE,
  encrypted_username TEXT NOT NULL,
  encrypted_password TEXT NOT NULL,
  agent_number TEXT,
  is_active BOOLEAN DEFAULT true,
  last_login_success TIMESTAMPTZ,
  last_login_error TEXT,
  login_failure_count INTEGER DEFAULT 0,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW(),
  UNIQUE(tenant_id, carrier_id)
);

ALTER TABLE ins_carrier_credentials ENABLE ROW LEVEL SECURITY;
CREATE POLICY "tenant_creds_policy" ON ins_carrier_credentials
  FOR ALL USING (tenant_id IN (SELECT tenant_id FROM users WHERE id = auth.uid()));

-- 3. QUOTE REQUESTS (solicitudes de cotización)
CREATE TABLE IF NOT EXISTS ins_quote_requests (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id UUID NOT NULL REFERENCES tenants(id),
  contact_id UUID,
  conversation_id UUID,
  insurance_line TEXT NOT NULL CHECK (insurance_line IN ('auto', 'vida', 'gastos_med
  status TEXT NOT NULL DEFAULT 'pending' CHECK (status IN ('pending', 'validating',
  client_name TEXT NOT NULL,
  client_phone TEXT,
  client_rfc TEXT,
  client_birthdate DATE,
  client_gender TEXT CHECK (client_gender IN ('M', 'F')),
  client_zip_code TEXT NOT NULL,
  client_state TEXT,
  client_city TEXT,
  vehicle_brand TEXT,
  vehicle_model TEXT,
  vehicle_year INTEGER,
  vehicle_version TEXT,
  vehicle_use TEXT DEFAULT 'particular',
  coverage_type TEXT DEFAULT 'amplia',
  carriers_targeted INTEGER DEFAULT 0,
```

```sql
  carriers_succeeded INTEGER DEFAULT 0,
  carriers_failed INTEGER DEFAULT 0,
  source TEXT DEFAULT 'whatsapp',
  raw_input TEXT,
  extracted_data JSONB,
  started_at TIMESTAMPTZ,
  first_result_at TIMESTAMPTZ,
  completed_at TIMESTAMPTZ,
  expires_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX IF NOT EXISTS idx_ins_qr_tenant ON ins_quote_requests(tenant_id);
CREATE INDEX IF NOT EXISTS idx_ins_qr_status ON ins_quote_requests(status);
CREATE INDEX IF NOT EXISTS idx_ins_qr_created ON ins_quote_requests(created_at DESC)

ALTER TABLE ins_quote_requests ENABLE ROW LEVEL SECURITY;
CREATE POLICY "tenant_quotes_policy" ON ins_quote_requests
  FOR ALL USING (tenant_id IN (SELECT tenant_id FROM users WHERE id = auth.uid()));

-- 4. QUOTES (cotizaciones individuales por carrier)
CREATE TABLE IF NOT EXISTS ins_quotes (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  quote_request_id UUID NOT NULL REFERENCES ins_quote_requests(id) ON DELETE CASCADE
  tenant_id UUID NOT NULL REFERENCES tenants(id),
  carrier_id UUID NOT NULL REFERENCES ins_carriers(id),
  status TEXT NOT NULL DEFAULT 'pending' CHECK (status IN ('pending', 'running', 'su
  annual_premium DECIMAL(12,2),
  monthly_premium DECIMAL(12,2),
  deductible_amount DECIMAL(12,2),
  deductible_percentage DECIMAL(5,2),
  coverages JSONB,
  quote_number TEXT,
  valid_until DATE,
  pdf_url TEXT,
  screenshot_url TEXT,
  duration_ms INTEGER,
  rank_position INTEGER,
  rank_score DECIMAL(5,2),
  error_message TEXT,
  error_type TEXT,
  retry_count INTEGER DEFAULT 0,
  started_at TIMESTAMPTZ,
```

```sql
  completed_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX IF NOT EXISTS idx_ins_q_request ON ins_quotes(quote_request_id);
CREATE INDEX IF NOT EXISTS idx_ins_q_tenant ON ins_quotes(tenant_id);
CREATE INDEX IF NOT EXISTS idx_ins_q_status ON ins_quotes(status);

ALTER TABLE ins_quotes ENABLE ROW LEVEL SECURITY;
CREATE POLICY "tenant_indiv_quotes_policy" ON ins_quotes
  FOR ALL USING (tenant_id IN (SELECT tenant_id FROM users WHERE id = auth.uid()));

-- 5. POLICIES (pólizas emitidas)
CREATE TABLE IF NOT EXISTS ins_policies (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id UUID NOT NULL REFERENCES tenants(id),
  contact_id UUID,
  carrier_id UUID NOT NULL REFERENCES ins_carriers(id),
  quote_id UUID REFERENCES ins_quotes(id),
  policy_number TEXT NOT NULL,
  insurance_line TEXT NOT NULL,
  status TEXT NOT NULL DEFAULT 'active' CHECK (status IN ('active', 'pending_payment
  start_date DATE NOT NULL,
  end_date DATE NOT NULL,
  total_premium DECIMAL(12,2),
  payment_frequency TEXT DEFAULT 'anual',
  next_payment_date DATE,
  payment_status TEXT DEFAULT 'current',
  risk_data JSONB,
  policy_pdf_url TEXT,
  commission_percentage DECIMAL(5,2),
  commission_amount DECIMAL(12,2),
  auto_renew BOOLEAN DEFAULT true,
  renewal_quote_id UUID,
  renewal_notified BOOLEAN DEFAULT false,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX IF NOT EXISTS idx_ins_pol_tenant ON ins_policies(tenant_id);
CREATE INDEX IF NOT EXISTS idx_ins_pol_end ON ins_policies(end_date);
CREATE INDEX IF NOT EXISTS idx_ins_pol_status ON ins_policies(status);
```

```sql
ALTER TABLE ins_policies ENABLE ROW LEVEL SECURITY;
CREATE POLICY "tenant_policies_policy" ON ins_policies
  FOR ALL USING (tenant_id IN (SELECT tenant_id FROM users WHERE id = auth.uid()));

-- 6. POLICY PAYMENTS
CREATE TABLE IF NOT EXISTS ins_policy_payments (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id UUID NOT NULL REFERENCES tenants(id),
  policy_id UUID NOT NULL REFERENCES ins_policies(id) ON DELETE CASCADE,
  payment_number INTEGER,
  amount DECIMAL(12,2) NOT NULL,
  due_date DATE NOT NULL,
  paid_date DATE,
  status TEXT NOT NULL DEFAULT 'pending' CHECK (status IN ('pending', 'paid', 'overd
  verified_from_portal BOOLEAN DEFAULT false,
  portal_check_at TIMESTAMPTZ,
  reminder_sent BOOLEAN DEFAULT false,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX IF NOT EXISTS idx_ins_pay_policy ON ins_policy_payments(policy_id);
CREATE INDEX IF NOT EXISTS idx_ins_pay_due ON ins_policy_payments(due_date);

ALTER TABLE ins_policy_payments ENABLE ROW LEVEL SECURITY;
CREATE POLICY "tenant_payments_policy" ON ins_policy_payments
  FOR ALL USING (tenant_id IN (SELECT tenant_id FROM users WHERE id = auth.uid()));

-- 7. CARRIER HEALTH LOG
CREATE TABLE IF NOT EXISTS ins_carrier_health_log (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  carrier_id UUID NOT NULL REFERENCES ins_carriers(id),
  check_type TEXT NOT NULL,
  status TEXT NOT NULL,
  response_time_ms INTEGER,
  error_message TEXT,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX IF NOT EXISTS idx_ins_health_carrier ON ins_carrier_health_log(carrier_

-- 8. QUOTE CACHE
CREATE TABLE IF NOT EXISTS ins_quote_cache (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  cache_key TEXT UNIQUE NOT NULL,
```

```sql
  carrier_id UUID NOT NULL REFERENCES ins_carriers(id),
  quote_data JSONB NOT NULL,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  expires_at TIMESTAMPTZ NOT NULL
);

CREATE INDEX IF NOT EXISTS idx_ins_cache_key ON ins_quote_cache(cache_key);
CREATE INDEX IF NOT EXISTS idx_ins_cache_exp ON ins_quote_cache(expires_at);

-- Vista: pólizas próximas a renovar
CREATE OR REPLACE VIEW ins_v_near_renewal AS
SELECT p.*, c.name as carrier_name, c.slug as carrier_slug,
  (p.end_date - CURRENT_DATE) as days_to_renewal
FROM ins_policies p
JOIN ins_carriers c ON p.carrier_id = c.id
WHERE p.status = 'active'
  AND (p.end_date - CURRENT_DATE) BETWEEN 0 AND 30
ORDER BY (p.end_date - CURRENT_DATE) ASC;

-- Vista: health dashboard
CREATE OR REPLACE VIEW ins_v_carrier_health AS
SELECT c.id, c.name, c.slug, c.health_status, c.failure_rate_24h,
  COUNT(CASE WHEN h.status = 'success' AND h.created_at > NOW() - INTERVAL '24 hours
  COUNT(CASE WHEN h.status != 'success' AND h.created_at > NOW() - INTERVAL '24 hour
  ROUND(AVG(CASE WHEN h.created_at > NOW() - INTERVAL '24 hours' THEN h.response_tim
FROM ins_carriers c
LEFT JOIN ins_carrier_health_log h ON c.id = h.carrier_id
GROUP BY c.id;

-- Función: limpiar cache expirado (llamar via cron)
CREATE OR REPLACE FUNCTION ins_cleanup_cache() RETURNS INTEGER AS $$
DECLARE deleted_count INTEGER;
BEGIN
  DELETE FROM ins_quote_cache WHERE expires_at < NOW();
  GET DIAGNOSTICS deleted_count = ROW_COUNT;
  RETURN deleted_count;
END;
$$ LANGUAGE plpgsql;
```

Ejecutar el SQL en Supabase SQL Editor.

Checkpoint Fase 1:

- [ ] `SELECT count(*) FROM ins_carriers;` retorna 15
- [ ] `SELECT * FROM ins_v_carrier_health;` funciona sin error
- [ ] `SELECT * FROM ins_v_near_renewal;` funciona sin error (0 rows está bien)
- [ ] Todas las tablas tienen RLS habilitado

FASE 2: Credential Vault — Encriptación AES-256-GCM

Objetivo: Sistema para encriptar/desencriptar credenciales de portales de aseguradoras.

Dependencias: Fase 0.

Qué pedir al usuario:

- [ ] "Genera tu encryption key ejecutando `openssl rand -hex 32` en terminal y agrégala como CREDENTIAL_ENCRYPTION_KEY en .env.local"

Crear `src/lib/insurance/credential-vault.ts`:

```typescript
```

```
import { randomBytes, createCipheriv, createDecipheriv } from 'crypto'

const ALGORITHM = 'aes-256-gcm'
const IV_LENGTH = 12
const AUTH_TAG_LENGTH = 16

function getKey(): Buffer {
  const key = process.env.CREDENTIAL_ENCRYPTION_KEY
  if (!key || key.length !== 64) {
    throw new Error('CREDENTIAL_ENCRYPTION_KEY must be a 64-char hex string (32 byte
  }
  return Buffer.from(key, 'hex')
}

export function encryptCredential(plaintext: string): string {
  const key = getKey()
  const iv = randomBytes(IV_LENGTH)
  const cipher = createCipheriv(ALGORITHM, key, iv, { authTagLength: AUTH_TAG_LENGTH
  const encrypted = Buffer.concat([cipher.update(plaintext, 'utf8'), cipher.final()]
  const authTag = cipher.getAuthTag()
  // Format: base64(iv + authTag + ciphertext)
  return Buffer.concat([iv, authTag, encrypted]).toString('base64')
}

export function decryptCredential(payload: string): string {
  const key = getKey()
  const buf = Buffer.from(payload, 'base64')
  const iv = buf.subarray(0, IV_LENGTH)
  const authTag = buf.subarray(IV_LENGTH, IV_LENGTH + AUTH_TAG_LENGTH)
  const ciphertext = buf.subarray(IV_LENGTH + AUTH_TAG_LENGTH)
  const decipher = createDecipheriv(ALGORITHM, key, iv, { authTagLength: AUTH_TAG_LE
  decipher.setAuthTag(authTag)
  return Buffer.concat([decipher.update(ciphertext), decipher.final()]).toString('ut
}
```

Checkpoint Fase 2:

- [ ] Test inline: agrega este test temporal al final del archivo y ejecútalo:

```typescript
```

```
// TEST (borrar después):
// const encrypted = encryptCredential('mi_password_secreto')
// console.log('encrypted:', encrypted)
// console.log('decrypted:', decryptCredential(encrypted))
// Debe imprimir: 'mi_password_secreto'
```

FASE 3: Circuit Breaker con Redis

Objetivo: Sistema que detecta cuando un portal de aseguradora está caído y deja de intentar.

Dependencias: Fase 0.

Qué pedir al usuario:

- [ ] "Dame tu UPSTASH_REDIS_REST_URL y UPSTASH_REDIS_REST_TOKEN"

Crear `src/lib/insurance/circuit-breaker.ts`**:**

```typescript
```

```
import { Redis } from '@upstash/redis'
import { CIRCUIT_BREAKER_THRESHOLD, CIRCUIT_BREAKER_TIMEOUT_MS } from './constants'

const redis = new Redis({
  url: process.env.UPSTASH_REDIS_REST_URL!,
  token: process.env.UPSTASH_REDIS_REST_TOKEN!,
})

interface CircuitState {
  failures: number
  successes: number
  total: number
  state: 'closed' | 'open' | 'half_open'
  last_failure_at: number
  opened_at: number
}

const DEFAULT_STATE: CircuitState = {
  failures: 0, successes: 0, total: 0,
  state: 'closed', last_failure_at: 0, opened_at: 0,
}

function getKey(carrierSlug: string): string {
  return `ins:cb:${carrierSlug}`
}

export async function isCircuitOpen(carrierSlug: string): Promise<boolean> {
  const state = await redis.get<CircuitState>(getKey(carrierSlug))
  if (!state) return false

  if (state.state === 'open') {
    // Check if timeout has passed → transition to half_open
    if (Date.now() - state.opened_at > CIRCUIT_BREAKER_TIMEOUT_MS) {
      await redis.set(getKey(carrierSlug), { ...state, state: 'half_open' }, { ex: 8
      return false // Allow one probe request
    }
    return true // Still open, reject
  }

  return false
}

export async function recordSuccess(carrierSlug: string): Promise<void> {
```

```
  const key = getKey(carrierSlug)
  const state = (await redis.get<CircuitState>(key)) || { ...DEFAULT_STATE }

  state.successes++
  state.total++

  // If half_open and success → close circuit
  if (state.state === 'half_open') {
    state.state = 'closed'
    state.failures = 0
  }

  await redis.set(key, state, { ex: 86400 })
}

export async function recordFailure(carrierSlug: string): Promise<void> {
  const key = getKey(carrierSlug)
  const state = (await redis.get<CircuitState>(key)) || { ...DEFAULT_STATE }

  state.failures++
  state.total++
  state.last_failure_at = Date.now()

  // If failures exceed threshold → open circuit
  if (state.failures >= CIRCUIT_BREAKER_THRESHOLD) {
    state.state = 'open'
    state.opened_at = Date.now()
  }

  await redis.set(key, state, { ex: 86400 })
}

export async function getCircuitState(carrierSlug: string): Promise<CircuitState> {
  return (await redis.get<CircuitState>(getKey(carrierSlug))) || { ...DEFAULT_STATE
}
```

Checkpoint Fase 3:

- [ ] No errors de TypeScript
- [ ] Redis conecta (lo probaremos en integración en Fase 5)

FASE 4: API Routes — Endpoints del Módulo

Objetivo: Crear TODOS los endpoints de la API de seguros.

Dependencias: Fases 1, 2, 3.

Crear `src/app/api/insurance/carriers/route.ts`:

```typescript
import { NextRequest, NextResponse } from 'next/server'
import { createRouteHandlerClient } from '@supabase/auth-helpers-nextjs'
import { cookies } from 'next/headers'

export async function GET() {
  const supabase = createRouteHandlerClient({ cookies })
  const { data: { user } } = await supabase.auth.getUser()
  if (!user) return NextResponse.json({ error: 'Unauthorized' }, { status: 401 })

  const { data: carriers, error } = await supabase
    .from('ins_carriers')
    .select('*')
    .eq('is_active', true)
    .order('market_share_auto', { ascending: false })

  if (error) return NextResponse.json({ error: error.message }, { status: 500 })
  return NextResponse.json(carriers)
}
```

Crear `src/app/api/insurance/credentials/route.ts`:

```typescript
```

```
import { NextRequest, NextResponse } from 'next/server'
import { createRouteHandlerClient } from '@supabase/auth-helpers-nextjs'
import { cookies } from 'next/headers'
import { encryptCredential } from '@/lib/insurance/credential-vault'

export async function POST(req: NextRequest) {
  const supabase = createRouteHandlerClient({ cookies })
  const { data: { user } } = await supabase.auth.getUser()
  if (!user) return NextResponse.json({ error: 'Unauthorized' }, { status: 401 })

  const { carrier_id, username, password, agent_number } = await req.json()

  if (!carrier_id || !username || !password) {
    return NextResponse.json({ error: 'carrier_id, username, password required' }, {
  }

  // Get tenant_id from user
  const { data: userRow } = await supabase
    .from('users')
    .select('tenant_id')
    .eq('id', user.id)
    .single()

  if (!userRow) return NextResponse.json({ error: 'User not found' }, { status: 404

  const encrypted_username = encryptCredential(username)
  const encrypted_password = encryptCredential(password)

  const { data, error } = await supabase
    .from('ins_carrier_credentials')
    .upsert({
      tenant_id: userRow.tenant_id,
      carrier_id,
      encrypted_username,
      encrypted_password,
      agent_number,
      is_active: true,
      login_failure_count: 0,
    }, { onConflict: 'tenant_id,carrier_id' })
    .select('id, carrier_id, agent_number, is_active')
    .single()

  if (error) return NextResponse.json({ error: error.message }, { status: 500 })
```

```
  return NextResponse.json(data)
}

export async function GET() {
  const supabase = createRouteHandlerClient({ cookies })
  const { data: { user } } = await supabase.auth.getUser()
  if (!user) return NextResponse.json({ error: 'Unauthorized' }, { status: 401 })

  const { data: userRow } = await supabase
    .from('users')
    .select('tenant_id')
    .eq('id', user.id)
    .single()

  if (!userRow) return NextResponse.json({ error: 'User not found' }, { status: 404

  // Return credentials WITHOUT decrypted values (only metadata)
  const { data, error } = await supabase
    .from('ins_carrier_credentials')
    .select('id, carrier_id, agent_number, is_active, last_login_success, login_fail
    .eq('tenant_id', userRow.tenant_id)

  if (error) return NextResponse.json({ error: error.message }, { status: 500 })
  return NextResponse.json(data)
}
```

Crear `src/app/api/insurance/quote/route.ts`:

```typescript
```

```
import { NextRequest, NextResponse } from 'next/server'
import { createRouteHandlerClient } from '@supabase/auth-helpers-nextjs'
import { cookies } from 'next/headers'
import { Client } from '@upstash/qstash'
import { decryptCredential } from '@/lib/insurance/credential-vault'
import { isCircuitOpen } from '@/lib/insurance/circuit-breaker'
import type { QuoteRequestInput, WorkerQuotePayload } from '@/lib/insurance/types'

const qstash = new Client({ token: process.env.QSTASH_TOKEN! })

export async function POST(req: NextRequest) {
  const supabase = createRouteHandlerClient({ cookies })
  const { data: { user } } = await supabase.auth.getUser()
  if (!user) return NextResponse.json({ error: 'Unauthorized' }, { status: 401 })

  const input: QuoteRequestInput = await req.json()

  // Get tenant
  const { data: userRow } = await supabase
    .from('users')
    .select('tenant_id')
    .eq('id', user.id)
    .single()

  if (!userRow) return NextResponse.json({ error: 'User not found' }, { status: 404

  const tenantId = userRow.tenant_id

  // 1. Create quote request
  const { data: quoteReq, error: qrErr } = await supabase
    .from('ins_quote_requests')
    .insert({
      tenant_id: tenantId,
      contact_id: input.contact_id,
      conversation_id: input.conversation_id,
      insurance_line: input.insurance_line,
      client_name: input.client.name,
      client_phone: input.client.phone,
      client_rfc: input.client.rfc,
      client_birthdate: input.client.birthdate,
      client_gender: input.client.gender,
      client_zip_code: input.client.zip_code,
      vehicle_brand: input.vehicle?.brand,
```

```
    vehicle_model: input.vehicle?.model,
    vehicle_year: input.vehicle?.year,
    vehicle_version: input.vehicle?.version,
    vehicle_use: input.vehicle?.use,
    coverage_type: input.coverage_type,
    status: 'quoting',
    source: input.source,
    raw_input: input.raw_input,
    started_at: new Date().toISOString(),
  })
  .select('id')
  .single()

if (qrErr) return NextResponse.json({ error: qrErr.message }, { status: 500 })

const requestId = quoteReq.id

// 2. Get carriers with active credentials for this insurance line
const { data: creds } = await supabase
  .from('ins_carrier_credentials')
  .select('*, ins_carriers(*)')
  .eq('tenant_id', tenantId)
  .eq('is_active', true)

const eligibleCarriers = (creds || []).filter(c =>
  c.ins_carriers.is_active &&
  c.ins_carriers.supported_lines.includes(input.insurance_line) &&
  c.ins_carriers.health_status !== 'down'
)

// 3. Create individual quote records
const quoteRecords = eligibleCarriers.map(c => ({
  quote_request_id: requestId,
  tenant_id: tenantId,
  carrier_id: c.carrier_id,
  status: 'pending' as const,
}))

if (quoteRecords.length > 0) {
  await supabase.from('ins_quotes').insert(quoteRecords)
}

// Update targeted count
await supabase
```

```
  .from('ins_quote_requests')
  .update({ carriers_targeted: eligibleCarriers.length })
  .eq('id', requestId)

// 4. Fan-out via QStash — one job per carrier
const workerUrl = process.env.INSURANCE_WORKER_URL
const appUrl = process.env.APP_URL || 'https://tu-app.vercel.app'

const dispatches = await Promise.allSettled(
  eligibleCarriers.map(async (cred) => {
    // Check circuit breaker
    if (await isCircuitOpen(cred.ins_carriers.slug)) {
      await supabase.from('ins_quotes').update({
        status: 'skipped', error_message: 'Circuit breaker open', error_type: 'cir
        completed_at: new Date().toISOString(),
      }).eq('quote_request_id', requestId).eq('carrier_id', cred.carrier_id)
      return
    }

    // Decrypt credentials
    const payload: WorkerQuotePayload = {
      request_id: requestId,
      tenant_id: tenantId,
      carrier_slug: cred.ins_carriers.slug,
      carrier_portal_url: cred.ins_carriers.portal_url,
      carrier_portal_type: cred.ins_carriers.portal_type,
      insurance_line: input.insurance_line,
      client_data: input.client,
      vehicle_data: input.vehicle,
      coverage_type: input.coverage_type,
      credentials: {
        username: decryptCredential(cred.encrypted_username),
        password: decryptCredential(cred.encrypted_password),
        agent_number: cred.agent_number || undefined,
      },
    }

    // Dispatch to worker via QStash
    return qstash.publishJSON({
      url: `${workerUrl}/quote`,
      body: payload,
      retries: 2,
      callback: `${appUrl}/api/insurance/callback`,
      failureCallback: `${appUrl}/api/insurance/callback`,
```

```
        headers: {
          'x-worker-secret': process.env.INSURANCE_WORKER_SECRET!,
        },
      })
    })
  )

  return NextResponse.json({
    request_id: requestId,
    carriers_targeted: eligibleCarriers.length,
    status: 'quoting',
    stream_url: `/api/insurance/stream?id=${requestId}`,
  })
}
```

Crear `src/app/api/insurance/callback/route.ts`:

```typescript
```

```
import { NextRequest, NextResponse } from 'next/server'
import { verifySignatureAppRouter } from '@upstash/qstash/nextjs'
import { createClient } from '@supabase/supabase-js'
import { Redis } from '@upstash/redis'
import { recordSuccess, recordFailure } from '@/lib/insurance/circuit-breaker'
import type { QuoteResult } from '@/lib/insurance/types'

const supabase = createClient(
  process.env.NEXT_PUBLIC_SUPABASE_URL!,
  process.env.SUPABASE_SERVICE_ROLE_KEY!
)

const redis = new Redis({
  url: process.env.UPSTASH_REDIS_REST_URL!,
  token: process.env.UPSTASH_REDIS_REST_TOKEN!,
})

async function handler(req: NextRequest) {
  const body = await req.json()

  // QStash wraps the worker response — extract it
  const result: QuoteResult = body.body || body

  const { data: carrier } = await supabase
    .from('ins_carriers')
    .select('id')
    .eq('slug', result.carrier_slug)
    .single()

  if (!carrier) return NextResponse.json({ error: 'Carrier not found' }, { status: 4

  // Update individual quote
  await supabase
    .from('ins_quotes')
    .update({
      status: result.status === 'success' ? 'success' : result.status === 'timeout'
      annual_premium: result.annual_premium,
      monthly_premium: result.monthly_premium,
      deductible_amount: result.deductible_amount,
      deductible_percentage: result.deductible_percentage,
      coverages: result.coverages,
      quote_number: result.quote_number,
      valid_until: result.valid_until,
```

```
    pdf_url: result.pdf_url,
    screenshot_url: result.screenshot_url,
    duration_ms: result.duration_ms,
    error_message: result.error_message,
    error_type: result.error_type,
    completed_at: new Date().toISOString(),
  })
  .eq('quote_request_id', result.request_id || body.request_id)
  .eq('carrier_id', carrier.id)

// Update circuit breaker
if (result.status === 'success') {
  await recordSuccess(result.carrier_slug)
} else {
  await recordFailure(result.carrier_slug)
}

// Check if all carriers done → finalize
const requestId = result.request_id || body.request_id
const { data: allQuotes } = await supabase
  .from('ins_quotes')
  .select('status, annual_premium, carrier_id, ins_carriers(name, slug)')
  .eq('quote_request_id', requestId)

const pending = (allQuotes || []).filter(q => q.status === 'pending' || q.status =
const succeeded = (allQuotes || []).filter(q => q.status === 'success')
const failed = (allQuotes || []).filter(q => ['error', 'timeout', 'skipped', 'decl

// Publish progress to Redis for SSE consumers
const progress = {
  request_id: requestId,
  total: (allQuotes || []).length,
  completed: succeeded.length,
  failed: failed.length,
  status: pending.length === 0 ? 'complete' : 'partial',
  results: succeeded.map(q => ({
    carrier_name: (q as any).ins_carriers?.name,
    carrier_slug: (q as any).ins_carriers?.slug,
    annual_premium: q.annual_premium,
  })).sort((a, b) => (a.annual_premium || Infinity) - (b.annual_premium || Infinit
  best_price: succeeded.length > 0
    ? Math.min(...succeeded.map(q => q.annual_premium || Infinity))
    : null,
}
```

```
  await redis.set(`ins:progress:${requestId}`, JSON.stringify(progress), { ex: 3600

  // If all done → finalize
  if (pending.length === 0) {
    // Rank quotes
    const ranked = succeeded
      .sort((a, b) => (a.annual_premium || Infinity) - (b.annual_premium || Infinity

    for (let i = 0; i < ranked.length; i++) {
      await supabase.from('ins_quotes').update({
        rank_position: i + 1,
      }).eq('quote_request_id', requestId).eq('carrier_id', ranked[i].carrier_id)
    }

    await supabase.from('ins_quote_requests').update({
      status: 'complete',
      carriers_succeeded: succeeded.length,
      carriers_failed: failed.length,
      completed_at: new Date().toISOString(),
      expires_at: new Date(Date.now() + 4 * 60 * 60 * 1000).toISOString(),
    }).eq('id', requestId)
  } else {
    await supabase.from('ins_quote_requests').update({
      status: 'partial',
      carriers_succeeded: succeeded.length,
      carriers_failed: failed.length,
    }).eq('id', requestId)
  }

  return NextResponse.json({ ok: true })
}

export const POST = verifySignatureAppRouter(handler)
```

Crear `src/app/api/insurance/stream/route.ts` (SSE progressive delivery):

```typescript
```

```
import { NextRequest } from 'next/server'
import { Redis } from '@upstash/redis'

const redis = new Redis({
  url: process.env.UPSTASH_REDIS_REST_URL!,
  token: process.env.UPSTASH_REDIS_REST_TOKEN!,
})

export const dynamic = 'force-dynamic'
export const runtime = 'nodejs'

export async function GET(req: NextRequest) {
  const requestId = req.nextUrl.searchParams.get('id')
  if (!requestId) {
    return new Response('Missing id parameter', { status: 400 })
  }

  const encoder = new TextEncoder()
  let closed = false

  const stream = new ReadableStream({
    start(controller) {
      function send(event: string, data: unknown) {
        if (closed) return
        try {
          controller.enqueue(encoder.encode(`event: ${event}\ndata: ${JSON.stringify
        } catch { closed = true }
      }

      send('connected', { request_id: requestId })

      let lastCompleted = 0

      const poll = setInterval(async () => {
        if (closed) { clearInterval(poll); return }

        try {
          const raw = await redis.get(`ins:progress:${requestId}`)
          if (!raw) return

          const progress = typeof raw === 'string' ? JSON.parse(raw) : raw

          // Only send if new results arrived
```

```
        if (progress.completed > lastCompleted || progress.failed > 0) {
          lastCompleted = progress.completed
          send('progress', progress)
        }

        if (progress.status === 'complete') {
          send('complete', progress)
          clearInterval(poll)
          clearInterval(heartbeat)
          setTimeout(() => { closed = true; controller.close() }, 500)
        }
      } catch (err) {
        console.error('SSE poll error:', err)
      }
    }, 1500) // Poll every 1.5s

    const heartbeat = setInterval(() => {
      if (closed) { clearInterval(heartbeat); return }
      try { controller.enqueue(encoder.encode(': heartbeat\n\n')) } catch { closed
    }, 15000)

    req.signal.addEventListener('abort', () => {
      closed = true
      clearInterval(poll)
      clearInterval(heartbeat)
      try { controller.close() } catch {}
    })
  },
})

return new Response(stream, {
  headers: {
    'Content-Type': 'text/event-stream',
    'Cache-Control': 'no-cache, no-transform',
    'Connection': 'keep-alive',
    'X-Accel-Buffering': 'no',
  },
})
}
```

Crear `src/app/api/insurance/quote/[id]/route.ts`:

```typescript
```

```
import { NextRequest, NextResponse } from 'next/server'
import { createRouteHandlerClient } from '@supabase/auth-helpers-nextjs'
import { cookies } from 'next/headers'

export async function GET(req: NextRequest, { params }: { params: { id: string } })
  const supabase = createRouteHandlerClient({ cookies })
  const { data: { user } } = await supabase.auth.getUser()
  if (!user) return NextResponse.json({ error: 'Unauthorized' }, { status: 401 })

  const { data: quoteReq, error } = await supabase
    .from('ins_quote_requests')
    .select('*, ins_quotes(*, ins_carriers(name, slug, logo_url))')
    .eq('id', params.id)
    .single()

  if (error) return NextResponse.json({ error: error.message }, { status: 500 })
  if (!quoteReq) return NextResponse.json({ error: 'Not found' }, { status: 404 })

  return NextResponse.json(quoteReq)
}
```

Checkpoint Fase 4:

- [ ] `npm run build` pasa sin errores
- [ ] Todos los archivos de API routes creados y sin errores de TypeScript
- [ ] Imports de credential-vault y circuit-breaker resuelven correctamente

---

FASE 5: Python Workers — Browser Automation Engine

Objetivo: Crear el servicio de Python que ejecuta la automation real en portales de aseguradoras.

Dependencias: Fases 0-4.

Qué pedir al usuario:

- [ ] "¿Tienes Docker instalado? Lo necesitamos para los workers."
- [ ] "¿Tienes cuenta de Railway? (railway.app) Los workers se deployean ahí."

Crear directorio `workers/` en la raíz del proyecto:

`workers/Dockerfile`:

```dockerfile
```

```
FROM python:3.12-slim

# Install system dependencies for Playwright
RUN apt-get update && apt-get install -y \
    wget gnupg2 curl unzip \
    libnss3 libnspr4 libatk1.0-0 libatk-bridge2.0-0 \
    libcups2 libdrm2 libdbus-1-3 libxkbcommon0 \
    libatspi2.0-0 libxcomposite1 libxdamage1 libxfixes3 \
    libxrandr2 libgbm1 libpango-1.0-0 libcairo2 libasound2 \
    && rm -rf /var/lib/apt/lists/*

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Install Playwright browsers
RUN playwright install chromium

COPY . .

EXPOSE 8080

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8080"]
```

`workers/requirements.txt`:

```
fastapi==0.115.0
uvicorn[standard]==0.30.0
playwright==1.43.0
browser-use==0.12.5
httpx==0.27.0
pydantic==2.6.0
python-dotenv==1.0.0
supabase==2.4.0
langchain-openai==0.3.0
```

`workers/main.py`:

```python
```

```python
import os
import time
import asyncio
import logging
from fastapi import FastAPI, HTTPException, Header
from pydantic import BaseModel
from typing import Optional
from dotenv import load_dotenv

load_dotenv()

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger("insurance-worker")

app = FastAPI(title="Insurance Quote Workers")

WORKER_SECRET = os.getenv("INSURANCE_WORKER_SECRET", "")


class ClientData(BaseModel):
    name: str
    phone: Optional[str] = None
    email: Optional[str] = None
    rfc: Optional[str] = None
    birthdate: Optional[str] = None
    gender: Optional[str] = None
    zip_code: str


class VehicleData(BaseModel):
    brand: str
    model: str
    year: int
    version: Optional[str] = None
    use: str = "particular"


class Credentials(BaseModel):
    username: str
    password: str
    agent_number: Optional[str] = None
```

```python
class QuotePayload(BaseModel):
    request_id: str
    tenant_id: str
    carrier_slug: str
    carrier_portal_url: str
    carrier_portal_type: str
    insurance_line: str
    client_data: ClientData
    vehicle_data: Optional[VehicleData] = None
    coverage_type: Optional[str] = "amplia"
    credentials: Credentials


class QuoteResponse(BaseModel):
    request_id: str
    carrier_slug: str
    status: str  # success | error | timeout | declined
    annual_premium: Optional[float] = None
    monthly_premium: Optional[float] = None
    deductible_amount: Optional[float] = None
    deductible_percentage: Optional[float] = None
    coverages: Optional[list] = None
    quote_number: Optional[str] = None
    valid_until: Optional[str] = None
    pdf_url: Optional[str] = None
    screenshot_url: Optional[str] = None
    duration_ms: Optional[int] = None
    error_message: Optional[str] = None
    error_type: Optional[str] = None


@app.get("/health")
async def health():
    return {"status": "ok", "service": "insurance-workers"}


@app.post("/quote", response_model=QuoteResponse)
async def execute_quote(
    payload: QuotePayload,
    x_worker_secret: str = Header(default=""),
):
    if x_worker_secret != WORKER_SECRET:
        raise HTTPException(status_code=401, detail="Invalid worker secret")
```

```
    start_time = time.time()
    logger.info(f"Starting quote: {payload.carrier_slug} for request {payload.reques

    try:
        result = await run_browser_quote(payload)
        result.duration_ms = int((time.time() - start_time) * 1000)
        logger.info(f"Quote complete: {payload.carrier_slug} — {result.status} in {r
        return result
    except asyncio.TimeoutError:
        duration = int((time.time() - start_time) * 1000)
        logger.warning(f"Quote timeout: {payload.carrier_slug} after {duration}ms")
        return QuoteResponse(
            request_id=payload.request_id,
            carrier_slug=payload.carrier_slug,
            status="timeout",
            duration_ms=duration,
            error_message="Portal automation timed out",
            error_type="timeout",
        )
    except Exception as e:
        duration = int((time.time() - start_time) * 1000)
        logger.error(f"Quote error: {payload.carrier_slug} — {str(e)}")
        return QuoteResponse(
            request_id=payload.request_id,
            carrier_slug=payload.carrier_slug,
            status="error",
            duration_ms=duration,
            error_message=str(e),
            error_type="automation_error",
        )


async def run_browser_quote(payload: QuotePayload) -> QuoteResponse:
    """Execute browser automation to get a quote from a carrier portal."""
    from browser_use import Agent, Browser
    from langchain_openai import ChatOpenAI

    # Build the prompt for Browser Use
    prompt = build_quote_prompt(payload)

    llm = ChatOpenAI(
        model="gpt-4o-mini",
        api_key=os.getenv("OPENAI_API_KEY"),
    )
```

```
    browser = Browser(headless=True)

    agent = Agent(
        task=prompt,
        llm=llm,
        browser=browser,
        sensitive_data={
            "portal_username": payload.credentials.username,
            "portal_password": payload.credentials.password,
        },
    )

    try:
        history = await asyncio.wait_for(
            agent.run(max_steps=40),
            timeout=110  # 110 seconds (leave 10s buffer from 120s QStash timeout)
        )

        # Extract result from the last agent output
        final_result = history.final_result()

        if final_result:
            # Parse the extracted data
            return parse_agent_result(payload, final_result)
        else:
            return QuoteResponse(
                request_id=payload.request_id,
                carrier_slug=payload.carrier_slug,
                status="error",
                error_message="Agent completed but no result extracted",
                error_type="extraction_failed",
            )
    finally:
        await browser.close()


def build_quote_prompt(payload: QuotePayload) -> str:
    """Build the Browser Use agent prompt for insurance quoting."""
    v = payload.vehicle_data
    c = payload.client_data

    if payload.insurance_line == "auto" and v:
        return f"""
```

```
Go to {payload.carrier_portal_url}

Step 1 - Login:
- Enter username: {{portal_username}}
- Enter password: {{portal_password}}
- Click login/sign in button
- Wait for dashboard to load

Step 2 - Navigate to quote/cotizador:
- Find and click on "Nueva Cotización", "Cotizar", "Cotizador", or similar
- If there's a menu, look for "Auto" or "Vehículos"

Step 3 - Fill vehicle information:
- Brand/Marca: {v.brand}
- Model/Modelo: {v.model}
- Year/Año: {v.year}
- Version/Versión: {v.version or "Select the first/basic option available"}
- Use/Uso: {"Particular" if v.use == "particular" else "Comercial/Servicio"}

Step 4 - Fill driver/insured information:
- Name/Nombre: {c.name}
- Birth date/Fecha nacimiento: {c.birthdate or "01/01/1990"}
- Gender/Sexo: {"Masculino" if c.gender == "M" else "Femenino"}
- Zip code/Código postal: {c.zip_code}
- RFC: {c.rfc or "Leave empty if optional"}

Step 5 - Select coverage:
- Coverage type: {payload.coverage_type or "amplia"}
- Look for "Amplia", "Todo Riesgo", "Cobertura Amplia" and select it

Step 6 - Calculate/Quote:
- Click "Cotizar", "Calcular", or "Obtener Precio"
- Wait for results to load

Step 7 - Extract ALL of these values from the results page:
- Annual premium (prima anual) — the total yearly cost in MXN
- Monthly premium (prima mensual) — if available
- Deductible percentage and/or amount
- List of included coverages with their sums insured
- Quote/cotización number if shown
- Validity date if shown

IMPORTANT RULES:
- If a field is a dropdown and you can't find the exact option, pick the closest mat
```

```
- If asked to accept terms/conditions, accept them
- If there's a CAPTCHA, try to solve it
- If login fails, report error immediately
- NEVER close the browser until you've extracted all the data
- Return ALL extracted data in a clear format
"""
    return f"Navigate to {payload.carrier_portal_url} and extract insurance quote in


def parse_agent_result(payload: QuotePayload, result: str) -> QuoteResponse:
    """Parse the Browser Use agent's text output into structured data."""
    import re

    # Try to extract numbers from the result text
    def find_amount(text: str, patterns: list[str]) -> Optional[float]:
        for pattern in patterns:
            match = re.search(pattern, text, re.IGNORECASE)
            if match:
                num_str = match.group(1).replace(",", "").replace(" ", "")
                try:
                    return float(num_str)
                except ValueError:
                    continue
        return None

    annual = find_amount(result, [
        r'prima\s*anual[:\s]*\$?([\d,]+\.?\d*)',
        r'annual[:\s]*\$?([\d,]+\.?\d*)',
        r'total\s*anual[:\s]*\$?([\d,]+\.?\d*)',
        r'costo\s*anual[:\s]*\$?([\d,]+\.?\d*)',
        r'\$?([\d,]+\.?\d*)\s*(?:anual|/año|por año)',
    ])

    monthly = find_amount(result, [
        r'prima\s*mensual[:\s]*\$?([\d,]+\.?\d*)',
        r'mensual[:\s]*\$?([\d,]+\.?\d*)',
        r'\$?([\d,]+\.?\d*)\s*(?:mensual|/mes|por mes)',
    ])

    deductible = find_amount(result, [
        r'deducible[:\s]*\$?([\d,]+\.?\d*)',
        r'deductible[:\s]*\$?([\d,]+\.?\d*)',
    ])
```

```
    deductible_pct = find_amount(result, [
        r'deducible[:\s]*([\d]+\.?\d*)%',
        r'([\d]+\.?\d*)%\s*de\s*deducible',
    ])

    quote_num_match = re.search(r'(?:cotización|quote|número|folio)[:\s#]*([A-Z0-9-]
    quote_number = quote_num_match.group(1) if quote_num_match else None

    if annual:
        return QuoteResponse(
            request_id=payload.request_id,
            carrier_slug=payload.carrier_slug,
            status="success",
            annual_premium=annual,
            monthly_premium=monthly or (round(annual / 12, 2) if annual else None),
            deductible_amount=deductible,
            deductible_percentage=deductible_pct,
            quote_number=quote_number,
        )
    else:
        return QuoteResponse(
            request_id=payload.request_id,
            carrier_slug=payload.carrier_slug,
            status="error",
            error_message=f"Could not extract premium from result: {result[:500]}",
            error_type="extraction_failed",
        )
```

`workers/.env.example`:

```env
INSURANCE_WORKER_SECRET=
OPENAI_API_KEY=
SKYVERN_API_KEY=
```

Test local:

```bash
```

```
cd workers
docker build -t insurance-workers .
docker run -p 8080:8080 --env-file .env insurance-workers
# En otra terminal:
curl http://localhost:8080/health
# Esperado: {"status":"ok","service":"insurance-workers"}
```

Deploy a Railway:

```bash
cd workers
# En railway.app: crear nuevo proyecto, conectar GitHub repo, seleccionar carpeta wo
# O usar CLI:
railway login
railway init
railway up
# Copiar la URL pública y agregarla como INSURANCE_WORKER_URL en .env.local de Verce
```

Checkpoint Fase 5:

- [ ] Docker build exitoso
- [ ] Health check responde 200
- [ ] Railway deploy exitoso (URL pública funcionando)

FASE 6: WhatsApp Insurance Flow

Objetivo: Integrar el flujo de cotización conversacional con el webhook de WhatsApp existente.

Dependencias: Fases 4, 5.

Crear `src/lib/insurance/whatsapp-flow.ts`**:**

```typescript
```

```
import type { QuoteRequestInput } from './types'
import { INSURANCE_LINE_LABELS, COVERAGE_LABELS } from './constants'

const EXTRACTION_SYSTEM_PROMPT = `Eres un asistente de seguros mexicano. Extrae dato

Responde SOLO con JSON válido, sin markdown ni explicación:
{
  "intent": "quote" | "other",
  "insurance_line": "auto" | "vida" | "gastos_medicos" | "hogar" | null,
  "extracted": {
    "brand": "string o null",
    "model": "string o null",
    "year": "number o null",
    "version": "string o null",
    "use": "particular o comercial o null",
    "coverage": "amplia o limitada o rc o null",
    "name": "string o null",
    "birthdate": "DD/MM/YYYY o null",
    "gender": "M o F o null",
    "zip_code": "string 5 dígitos o null",
    "rfc": "string o null"
  },
  "missing": ["lista de campos que faltan para cotizar"],
  "next_question": "pregunta amigable en español para pedir los datos faltantes (máx
}`

export async function processInsuranceMessage(
  message: string,
  previousData: Record<string, unknown> = {},
): Promise<{
  reply: string
  readyToQuote: boolean
  quoteInput?: QuoteRequestInput
  collectedData: Record<string, unknown>
}> {
  // Call LLM to extract data
  const response = await fetch('https://openrouter.ai/api/v1/chat/completions', {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
      'Authorization': `Bearer ${process.env.OPENROUTER_API_KEY}`,
    },
    body: JSON.stringify({
```

```
    model: 'google/gemini-2.5-flash',
    messages: [
      { role: 'system', content: EXTRACTION_SYSTEM_PROMPT },
      { role: 'user', content: `Datos previos: ${JSON.stringify(previousData)}\n\n
    ],
    response_format: { type: 'json_object' },
    temperature: 0.1,
  }),
})

const data = await response.json()
let extraction: any

try {
  extraction = JSON.parse(data.choices[0].message.content)
} catch {
  return {
    reply: '¡Hola! ¿Quieres cotizar un seguro? Dime qué tipo (auto, vida, gastos m
    readyToQuote: false,
    collectedData: previousData,
  }
}

// Merge with previous data
const merged = { ...previousData }
for (const [key, val] of Object.entries(extraction.extracted || {})) {
  if (val !== null && val !== undefined) {
    merged[key] = val
  }
}

// Check if ready to quote
const missing = extraction.missing || []
if (missing.length === 0 && extraction.insurance_line) {
  // Build summary for confirmation
  const line = extraction.insurance_line
  const summary = buildSummary(line, merged)

  return {
    reply: `📋 Tengo todos los datos:\n\n${summary}\n\n¿Todo correcto? Responde *S
    readyToQuote: false, // Wait for confirmation
    collectedData: { ...merged, _awaiting_confirmation: true, _insurance_line: lin
  }
}
```

```
  // Check if user is confirming
  if (previousData._awaiting_confirmation && /^(si|sí|yes|ok|dale|va|cotiza)/i.test(
    const line = previousData._insurance_line as string

    const quoteInput: QuoteRequestInput = {
      insurance_line: line as any,
      client: {
        name: (merged.name as string) || '',
        zip_code: (merged.zip_code as string) || '',
        birthdate: merged.birthdate as string,
        gender: merged.gender as 'M' | 'F',
        rfc: merged.rfc as string,
      },
      vehicle: line === 'auto' ? {
        brand: (merged.brand as string) || '',
        model: (merged.model as string) || '',
        year: (merged.year as number) || 2024,
        version: merged.version as string,
        use: (merged.use as 'particular' | 'comercial') || 'particular',
      } : undefined,
      coverage_type: (merged.coverage as any) || 'amplia',
      source: 'whatsapp',
      raw_input: message,
    }

    return {
      reply: `⏳ ¡Listo! Cotizando con *15+ aseguradoras* simultáneamente...\n\nTe m
      readyToQuote: true,
      quoteInput,
      collectedData: merged,
    }
  }

  return {
    reply: extraction.next_question || '¿Podrías darme más detalles para cotizar tu
    readyToQuote: false,
    collectedData: merged,
  }
}

function buildSummary(line: string, data: Record<string, unknown>): string {
  if (line === 'auto') {
    return [
```

```
      `🚗 *Vehículo*: ${data.brand} ${data.model} ${data.year}`,
      data.version ? `   Versión: ${data.version}` : '',
      `📍 *CP*: ${data.zip_code}`,
      `👤 *Conductor*: ${data.name}`,
      `🛡️ *Cobertura*: ${COVERAGE_LABELS[(data.coverage as string) || 'amplia'] || '
    ].filter(Boolean).join('\n')
  }
  return `Tipo: ${INSURANCE_LINE_LABELS[line] || line}`
}

export function formatProgressMessage(progress: any): string {
  const { total, completed, failed, results, status } = progress

  if (completed === 0 && failed === 0) {
    return `⏳ Cotizando... (0/${total} aseguradoras)`
  }

  let msg = `📊 *${completed}/${total} cotizaciones listas*\n\n`

  const sorted = (results || []).sort((a: any, b: any) =>
    (a.annual_premium || Infinity) - (b.annual_premium || Infinity)
  )

  sorted.forEach((r: any, i: number) => {
    const medal = ['🥇', '🥈', '🥉'][i] || '▪️'
    const price = r.annual_premium
      ? `$${Number(r.annual_premium).toLocaleString('es-MX', { minimumFractionDigits
      : 'N/A'
    msg += `${medal} *${r.carrier_name}*: ${price} /año\n`
  })

  if (status === 'complete') {
    msg += `\n✅ *Cotización completa*`
    if (failed > 0) msg += ` (${failed} no disponibles)`
    msg += `\n\n¿Te interesa alguna? Te doy más detalles.`
  } else {
    const remaining = total - completed - failed
    msg += `\n⏳ _${remaining} aseguradoras cotizando..._`
  }

  return msg
}
```

Checkpoint Fase 6:

- [ ] Archivo creado sin errores TypeScript
- [ ] Función processInsuranceMessage exportada correctamente
- [ ] Función formatProgressMessage exportada correctamente

FASE 7: Vercel Cron Jobs — Background Agents

Objetivo: Configurar cron jobs para renovaciones y health checks.

Dependencias: Fase 4.

Crear `src/app/api/cron/insurance/route.ts`:

```typescript
```

```
import { NextRequest, NextResponse } from 'next/server'
import { createClient } from '@supabase/supabase-js'

const supabase = createClient(
  process.env.NEXT_PUBLIC_SUPABASE_URL!,
  process.env.SUPABASE_SERVICE_ROLE_KEY!
)

export async function GET(req: NextRequest) {
  // Verify cron secret (Vercel sets this automatically)
  const authHeader = req.headers.get('authorization')
  if (authHeader !== `Bearer ${process.env.CRON_SECRET}`) {
    return NextResponse.json({ error: 'Unauthorized' }, { status: 401 })
  }

  const results = {
    renewals_checked: 0,
    overdue_payments: 0,
    cache_cleaned: 0,
  }

  // 1. Check renewals (policies expiring in 30 days)
  const { data: nearRenewal } = await supabase
    .from('ins_v_near_renewal')
    .select('*')

  results.renewals_checked = nearRenewal?.length || 0

  // For each policy near renewal that hasn't been notified:
  for (const policy of nearRenewal || []) {
    if (!policy.renewal_notified && policy.days_to_renewal <= 30) {
      // Mark as notified (WhatsApp notification handled by existing atiende.ai flow
      await supabase.from('ins_policies')
        .update({ renewal_notified: true })
        .eq('id', policy.id)
    }
  }

  // 2. Mark overdue payments
  const { data: overdue } = await supabase
    .from('ins_policy_payments')
    .update({ status: 'overdue' })
    .lt('due_date', new Date().toISOString().split('T')[0])
```

```
    .eq('status', 'pending')
    .select('id')

  results.overdue_payments = overdue?.length || 0

  // 3. Clean expired cache
  const { data: cacheResult } = await supabase.rpc('ins_cleanup_cache')
  results.cache_cleaned = cacheResult || 0

  return NextResponse.json({ ok: true, ...results })
}
```

Agregar al `vercel.json` (crear si no existe):

```json
{
  "crons": [
    {
      "path": "/api/cron/insurance",
      "schedule": "0 14 * * *"
    }
  ]
}
```

(14:00 UTC = 8:00 AM Ciudad de México)

Checkpoint Fase 7:

- [ ] Archivo cron creado
- [ ] vercel.json configurado
- [ ] `curl localhost:3000/api/cron/insurance` funciona con el header correcto

FASE 8: Dashboard Frontend — Páginas Principales

Objetivo: Crear las páginas del dashboard de seguros.

Dependencias: Fase 4.

Crear `src/app/(dashboard)/insurance/page.tsx`:

```typescript
```

```
import { createServerComponentClient } from '@supabase/auth-helpers-nextjs'
import { cookies } from 'next/headers'

export default async function InsuranceDashboard() {
  const supabase = createServerComponentClient({ cookies })

  const { data: { user } } = await supabase.auth.getUser()
  if (!user) return <div>No autorizado</div>

  // Get tenant
  const { data: userRow } = await supabase.from('users').select('tenant_id').eq('id'
  const tenantId = userRow?.tenant_id

  // Stats
  const [quotesRes, policiesRes, carriersRes] = await Promise.all([
    supabase.from('ins_quote_requests').select('id', { count: 'exact' }).eq('tenant_
    supabase.from('ins_policies').select('id', { count: 'exact' }).eq('tenant_id', t
    supabase.from('ins_carrier_credentials').select('id', { count: 'exact' }).eq('te
  ])

  const totalQuotes = quotesRes.count || 0
  const activePolicies = policiesRes.count || 0
  const connectedCarriers = carriersRes.count || 0

  return (
    <div className="space-y-6">
      <h1 className="text-2xl font-bold">Módulo de Seguros</h1>

      <div className="grid grid-cols-1 md:grid-cols-3 gap-4">
        <div className="bg-white dark:bg-gray-800 p-6 rounded-lg shadow">
          <p className="text-sm text-gray-500">Cotizaciones Totales</p>
          <p className="text-3xl font-bold">{totalQuotes}</p>
        </div>
        <div className="bg-white dark:bg-gray-800 p-6 rounded-lg shadow">
          <p className="text-sm text-gray-500">Pólizas Activas</p>
          <p className="text-3xl font-bold">{activePolicies}</p>
        </div>
        <div className="bg-white dark:bg-gray-800 p-6 rounded-lg shadow">
          <p className="text-sm text-gray-500">Aseguradoras Conectadas</p>
          <p className="text-3xl font-bold">{connectedCarriers}/15</p>
        </div>
      </div>
```

```
      <div className="bg-white dark:bg-gray-800 p-6 rounded-lg shadow">
        <h2 className="text-lg font-semibold mb-4">Conectar Aseguradoras</h2>
        <p className="text-gray-500 mb-4">Agrega tus credenciales de agente para emp
        <a href="/insurance/carriers" className="bg-blue-600 text-white px-4 py-2 ro
          Configurar Aseguradoras →
        </a>
      </div>
    </div>
  )
}
```

Crear `src/app/(dashboard)/insurance/carriers/page.tsx`**:**

```typescript
```

```
'use client'
import { useState, useEffect } from 'react'

interface Carrier {
  id: string; name: string; slug: string; market_share_auto: number; health_status:
}

interface ConnectedCarrier {
  carrier_id: string; agent_number: string | null; is_active: boolean
  ins_carriers: { name: string; slug: string }
}

export default function CarriersPage() {
  const [carriers, setCarriers] = useState<Carrier[]>([])
  const [connected, setConnected] = useState<ConnectedCarrier[]>([])
  const [form, setForm] = useState({ carrier_id: '', username: '', password: '', age
  const [saving, setSaving] = useState(false)
  const [message, setMessage] = useState('')

  useEffect(() => {
    fetch('/api/insurance/carriers').then(r => r.json()).then(setCarriers)
    fetch('/api/insurance/credentials').then(r => r.json()).then(setConnected)
  }, [])

  const connectedIds = new Set(connected.map(c => c.carrier_id))

  async function handleSubmit(e: React.FormEvent) {
    e.preventDefault()
    setSaving(true)
    setMessage('')

    const res = await fetch('/api/insurance/credentials', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(form),
    })

    if (res.ok) {
      setMessage('✅ Credenciales guardadas (encriptadas)')
      setForm({ carrier_id: '', username: '', password: '', agent_number: '' })
      fetch('/api/insurance/credentials').then(r => r.json()).then(setConnected)
    } else {
      const err = await res.json()
```

```
    setMessage(`❌ Error: ${err.error}`)
  }
  setSaving(false)
}

return (
  <div className="space-y-6">
    <h1 className="text-2xl font-bold">Mis Aseguradoras</h1>

    {/* Connected carriers */}
    <div className="bg-white dark:bg-gray-800 p-6 rounded-lg shadow">
      <h2 className="text-lg font-semibold mb-4">Conectadas ({connected.length})</
      {connected.length === 0 ? (
        <p className="text-gray-500">Aún no has conectado ninguna aseguradora.</p>
      ) : (
        <div className="space-y-2">
          {connected.map(c => (
            <div key={c.carrier_id} className="flex items-center justify-between p
              <span className="font-medium">{c.ins_carriers.name}</span>
              <span className="text-green-600 text-sm">✓ Conectada</span>
            </div>
          ))}
        </div>
      )}
    </div>

    {/* Add new carrier */}
    <div className="bg-white dark:bg-gray-800 p-6 rounded-lg shadow">
      <h2 className="text-lg font-semibold mb-4">Agregar Aseguradora</h2>
      <form onSubmit={handleSubmit} className="space-y-4 max-w-md">
        <div>
          <label className="block text-sm font-medium mb-1">Aseguradora</label>
          <select
            value={form.carrier_id}
            onChange={e => setForm(f => ({ ...f, carrier_id: e.target.value }))}
            className="w-full border rounded p-2"
            required
          >
            <option value="">Seleccionar...</option>
            {carriers.filter(c => !connectedIds.has(c.id)).map(c => (
              <option key={c.id} value={c.id}>{c.name} ({c.market_share_auto}% mar
            ))}
          </select>
        </div>
```

```
          <div>
            <label className="block text-sm font-medium mb-1">Usuario del portal</la
            <input type="text" value={form.username} onChange={e => setForm(f => ({
              className="w-full border rounded p-2" required placeholder="Tu usuario
          </div>
          <div>
            <label className="block text-sm font-medium mb-1">Contraseña del portal<
            <input type="password" value={form.password} onChange={e => setForm(f =>
              className="w-full border rounded p-2" required placeholder="Se guardar
          </div>
          <div>
            <label className="block text-sm font-medium mb-1">Número de agente (opci
            <input type="text" value={form.agent_number} onChange={e => setForm(f =>
              className="w-full border rounded p-2" placeholder="Tu cédula CNSF" />
          </div>
          <button type="submit" disabled={saving}
            className="bg-blue-600 text-white px-6 py-2 rounded hover:bg-blue-700 di
            {saving ? 'Guardando...' : '🔐 Guardar credenciales'}
          </button>
          {message && <p className="text-sm mt-2">{message}</p>}
        </form>
        <p className="text-xs text-gray-400 mt-4">🔒 Tus credenciales se encriptan c
      </div>
    </div>
  )
}
```

Checkpoint Fase 8:

- [ ] `npm run build` pasa
- [ ] `/insurance` muestra el dashboard con stats
- [ ] `/insurance/carriers` muestra formulario para conectar aseguradoras
- [ ] Guardar credenciales funciona y muestra "✅ Guardadas"

FASE 9: Integración End-to-End

Objetivo: Conectar el flujo completo: WhatsApp → API → Workers → Callback → WhatsApp response.

Dependencias: Todas las fases anteriores.

Qué pedir al usuario:

- [ ] "Dame la URL de tu Railway worker (INSURANCE_WORKER_URL)"
- [ ] "¿Tienes credenciales de prueba de al menos UNA aseguradora?"
- [ ] "Dame tu QSTASH_TOKEN, QSTASH_CURRENT_SIGNING_KEY, QSTASH_NEXT_SIGNING_KEY"

Qué hacer:

1. Actualizar `.env.local` con todas las URLs y tokens reales
2. En el webhook handler de WhatsApp existente de atiende.ai, agregar detección de intent de seguros:

Localizar el archivo del webhook de WhatsApp existente (probablemente `src/app/api/webhooks/whatsapp/route.ts` o similar) y AGREGAR esta lógica:

```typescript
```

```
// AGREGAR al handler existente de mensajes de WhatsApp:
import { processInsuranceMessage, formatProgressMessage } from '@/lib/insurance/what

// Dentro del handler de mensajes entrantes, ANTES del flujo normal:
// Detectar si el mensaje es sobre seguros
const insuranceKeywords = ['seguro', 'cotizar seguro', 'póliza', 'aseguradora', 'cot
const isInsuranceQuery = insuranceKeywords.some(kw => messageText.toLowerCase().incl

if (isInsuranceQuery) {
  // Get or create insurance conversation state from Redis
  const stateKey = `ins:wa:${from}:state`
  const redis = new Redis({ url: process.env.UPSTASH_REDIS_REST_URL!, token: process
  const prevState = await redis.get(stateKey) as Record<string, unknown> || {}

  const result = await processInsuranceMessage(messageText, prevState)

  // Save state
  await redis.set(stateKey, result.collectedData, { ex: 3600 })

  // Send reply
  await sendWhatsAppMessage(from, result.reply)

  if (result.readyToQuote && result.quoteInput) {
    // Launch the quote via API
    const quoteRes = await fetch(`${process.env.APP_URL}/api/insurance/quote`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json', 'Authorization': `Bearer ${user
      body: JSON.stringify(result.quoteInput),
    })
    const quoteData = await quoteRes.json()

    // Start polling for results and sending progressive updates
    // This runs in the background
    pollAndDeliverResults(quoteData.request_id, from)
  }

  return // Don't process through normal atiende.ai flow
}
```

3. Agregar función de progressive delivery:

```typescript
async function pollAndDeliverResults(requestId: string, whatsappNumber: string) {
  const redis = new Redis({ url: process.env.UPSTASH_REDIS_REST_URL!, token: process

  let lastSent = 0
  const maxPolls = 60 // 60 * 2s = 2 minutes max

  for (let i = 0; i < maxPolls; i++) {
    await new Promise(resolve => setTimeout(resolve, 2000)) // 2s interval

    const raw = await redis.get(`ins:progress:${requestId}`)
    if (!raw) continue

    const progress = typeof raw === 'string' ? JSON.parse(raw) : raw

    // Only send if new results
    if (progress.completed > lastSent) {
      lastSent = progress.completed
      const msg = formatProgressMessage(progress)
      await sendWhatsAppMessage(whatsappNumber, msg)
    }

    if (progress.status === 'complete') {
      // Clear conversation state
      await redis.del(`ins:wa:${whatsappNumber}:state`)
      break
    }
  }
}
```

Checkpoint Fase 9:

- [ ] Enviar "Quiero cotizar seguro de auto" por WhatsApp → bot responde pidiendo datos
- [ ] Dar datos completos → bot lanza cotización
- [ ] Resultados llegan progresivamente por WhatsApp
- [ ] Dashboard muestra la cotización con resultados

FASE 10: Deploy a Vercel

Objetivo: Deploy completo a producción.

Dependencias: Todas las fases.

Qué pedir al usuario:

- [ ] "¿Todas las variables de entorno están en Vercel Dashboard?"

Pasos de deploy:

```bash
# 1. Verificar build local
npm run build

# 2. Verificar que vercel.json tiene el cron configurado

# 3. Push a GitHub (Vercel auto-deploy si ya está conectado)
git add .
git commit -m "feat: módulo de seguros agéntico"
git push origin main

# 4. O deploy manual:
npx vercel --prod

# 5. Configurar variables de entorno en Vercel Dashboard:
# - QSTASH_TOKEN
# - QSTASH_CURRENT_SIGNING_KEY
# - QSTASH_NEXT_SIGNING_KEY
# - UPSTASH_REDIS_REST_URL
# - UPSTASH_REDIS_REST_TOKEN
# - SKYVERN_API_KEY
# - INSURANCE_WORKER_URL (URL de Railway)
# - INSURANCE_WORKER_SECRET
# - CREDENTIAL_ENCRYPTION_KEY
# - APP_URL (tu URL de Vercel, ej: https://atiende-ai.vercel.app)
# - CRON_SECRET (genera con: openssl rand -hex 16)
```

Verificación post-deploy:

```bash
```

```
# Health check workers
curl https://TU-RAILWAY-URL.up.railway.app/health

# Health check app
curl https://TU-APP.vercel.app/api/insurance/carriers

# Test cron
curl -H "Authorization: Bearer TU_CRON_SECRET" https://TU-APP.vercel.app/api/cron/in
```

Checkpoint Final:

- [ ] App en Vercel respondiendo correctamente
- [ ] Workers en Railway respondiendo
- [ ] Carriers endpoint retorna 15 aseguradoras
- [ ] Dashboard de seguros visible en `/insurance`
- [ ] Formulario de credenciales funcional
- [ ] WhatsApp flow detecta "cotizar seguro" y responde
- [ ] Primera cotización completa ejecutada (aunque sea con 1 carrier)
- [ ] Cron job ejecutándose diariamente

CHECKLIST GLOBAL ANTES DE ENTREGAR AL PRIMER CLIENTE

- [ ] Al menos 3 aseguradoras conectadas con credenciales de prueba
- [ ] Cotización end-to-end por WhatsApp funcionando
- [ ] Progressive delivery mostrando resultados conforme llegan
- [ ] Dashboard con métricas actualizándose en real-time
- [ ] Credenciales encriptadas verificadas (decrypt funciona)
- [ ] Circuit breaker probado (simular falla → carrier se skipea)
- [ ] Cron de renovaciones ejecutando
- [ ] Zero errores en Sentry
- [ ] Build production sin warnings
- [ ] Variables de entorno en Vercel = todas configuradas
- [ ] Workers en Railway = healthy y respondiendo
- [ ] SSL/HTTPS en ambos servicios

QUÉ NECESITA EL USUARIO (SOLO ESTO):

1. **API Keys** — OpenRouter, Supabase, Upstash (Redis + QStash), Skyvern, Sentry
2. **Credenciales de portal** — De al menos 1 aseguradora (Qualitas es ideal por market share)
3. **Railway account** — Para deployear los Python workers
4. **OpenAI API Key** — Para Browser Use en los workers (usa gpt-4o-mini)
5. **2 minutos** — Para generar la CREDENTIAL_ENCRYPTION_KEY con `openssl rand -hex 32`

Todo lo demás lo construye Claude Code siguiendo esta guía.